



Divide and Conquer

ค่ายโอลิมปิกวิชาการ วิชาคอมพิวเตอร์ ค่ายที่ 2 วันที่ 5
วิทยากร - ผู้ช่วยศาสตราจารย์ ดร. อัครา ประโยชน์

CIS KMUTNB

1

หัวข้อ

- Introduction Discrete Mathematics
 - Recurrence Relation
 - Analysis of Algorithm Efficiency
- Divide and Conquer
 - Merge Sort
 - Integer Multiplication
 - Matrix Multiplication
 - Counting Inversion

CIS KMUTNB

Divide and Conquer

2

Recurrence Relations

นิยาม A recurrence relation for the sequence $\{a_n\}$ is an equation that expresses a_n in terms of one or more of the previous terms of the sequence, namely, $a_0, a_1, a_2, \dots, a_{n-1}$, for all integer n with $n \geq n_0$, where n_0 is a nonnegative integer.

- Ex 7. $a_n = a_{n-1} + 3$ for $n=1,2,3,\dots$, and $a_0 = 2$. What are a_1, a_2, a_3 ?
 $a_1 = a_0 + 3 = 5$
 $a_2 = a_1 + 3 = 8$
 $a_3 = a_2 + 3 = 11$
- Ex 8. $a_n = 1.02a_{n-1}$ for $n=1,2,3,\dots$, and $a_0 = 20000$. What is a_{20} ? 21

Modeling with recurrence relations

Rabbits and the Fibonacci Numbers

- A young pair of rabbits (one of each sex) is placed on an island.
- A pair of rabbits does not breed until they are 2 months old.
- After they are 2 months old, each pair of rabbits produces another pair each month.
- The sequence a_n satisfies the recurrence relation

$$a_n = a_{n-1} + a_{n-2}$$

for $n \geq 3$, $a_1 = 1$, $a_2 = 1$

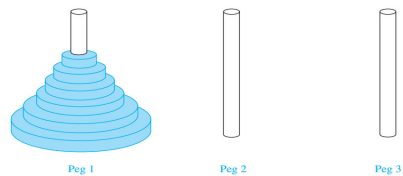
Reproducing pairs (at least two months old)	Young pairs (less than two months old)	Month	Reproducing pairs	Young pairs	Total pairs
		1	0	1	1
		2	0	1	1
		3	1	1	2
		4	1	2	3
		5	2	3	5
		6	3	5	8

The Tower of Hanoi

- Tower of Hanoi consists of three pegs mounted on board together with disks of different sizes.
- Initially these disks are placed on the first peg in order of size, with the largest on the bottom.
- Rules: disks to be moved one at a time from one peg to another as long as a disk is never placed on top of a smaller disk.
- Goal: all the disks on the second peg in order of size, with the largest on the bottom
- This shows that

$$H_n = 2H_{n-1} + 1$$

for $n \geq 2$, $H_1 = 1$



แบบฝึกหัด 1

- (Program) จงเขียนโปรแกรมแบบ recursive สำหรับปัญหาแต่ละข้อต่อไปนี้
 - A recursive algorithm for Computing $n!$
 - A recursive algorithm for Computing a^n
 - A Recursive Algorithm for Computing $\text{gcd}(a, b)$ (gcd = ห.ร.ม)
 - A Recursive Linear Search Algorithm
 - A Recursive Binary Search Algorithm
 - A Recursive Algorithm for Fibonacci Numbers
 - An Iterative Algorithm for Computing Fibonacci Number

Analysis of Algorithm Efficiency

1. Analysis Framework
2. Asymptotic notations and basic efficiency classes
3. Mathematical analysis of **non-recursive** algorithms
4. Mathematical analysis of **recursive** algorithms

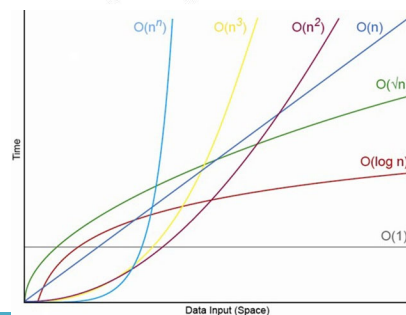
Analysis Framework

- Time efficiency
 - indicates how fast an algorithm in question runs
- Space efficiency
 - deals with the extra space the algorithm requires

Order of Growth

n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3.3	10^1	$3.3 \cdot 10^1$	10^2	10^3	10^3	$3.6 \cdot 10^6$
10^2	6.6	10^2	$6.6 \cdot 10^2$	10^4	10^6	$1.3 \cdot 10^{30}$	$9.3 \cdot 10^{157}$
10^3	10	10^3	$1.0 \cdot 10^4$	10^6	10^9		
10^4	13	10^4	$1.3 \cdot 10^5$	10^8	10^{12}		
10^5	17	10^5	$1.7 \cdot 10^6$	10^{10}	10^{15}		
10^6	20	10^6	$2.0 \cdot 10^7$	10^{12}	10^{18}		

Table 2.1 Values (some approximate) of several functions important for analysis of algorithms



CIS KMUTNB

Discrete mathematics

9

Best-case, average-case, worst-case

- นอกจาก input size แล้ว จำนวนการทำงานซ้ำของ basic operation อาจขึ้นกับลักษณะของ input ด้วย
 - Worst case: $W(n)$ – maximum over inputs of size n
 - Best case: $B(n)$ – minimum over inputs of size n
 - Average case: $A(n)$ – “average” over inputs of size n
 - Number of times the basic operation will be executed on typical input
 - NOT the average of worst and best case
 - Expected number of basic operations repetitions considered as a random variable under some assumption about the probability distribution of all possible inputs of size n

CIS KMUTNB

Discrete mathematics

10

Sequential Search

Algorithm SequentialSearch($A[0..n]$, K)

```

A[n] ← K
i ← 0
while A[i] ≠ K do
    i ← i+1
if i < n return i
else return -1

```

- *Problem*: กำหนดให้มีชุดข้อมูลใน array จำนวน n ชุด ให้หาชุดข้อมูลที่มีค่าเท่ากับ K
- *Algorithm*: ทำการตรวจสอบข้อมูลที่ละตัวไปเรื่อยๆ ว่ามีตัวใดมีค่าเท่ากับ K จนกว่าจะพบ (*successful search*) หรือจะหมดข้อมูลที่จะทำการค้นหา (*unsuccessful search*)
- Worst case : $C_w(n) = n$
- Best case : $C_b(n) = 1$
- Average case : $C_{avg}(n) = [1 \cdot \frac{p}{n} + 2 \cdot \frac{p}{n} + \dots + i \cdot \frac{p}{n} + \dots + n \cdot \frac{p}{n}] + n(1-p)$

$$= \frac{p(n+1)}{2} + n(1-p)$$

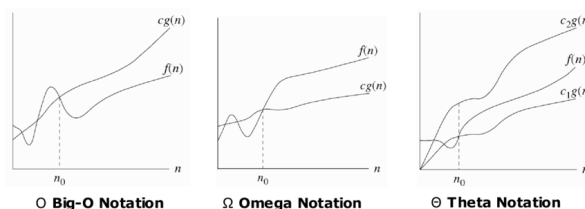
CIS KMUTNB

Discrete mathematics

11

Asymptotic growth rate

- A way of comparing functions that ignores constant factors and small input sizes
- $O(g(n))$: class of functions $f(n)$ that grow no faster than $g(n)$
- $\Theta(g(n))$: class of functions $f(n)$ that grow at same rate as $g(n)$
- $\Omega(g(n))$: class of functions $f(n)$ that grow at least as fast as $g(n)$



CIS KMUTNB

Discrete mathematics

12

Big-oh

$$t(n) \in O(g(n))$$

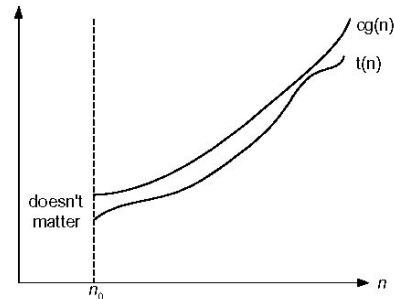
- if $t(n)$ is bounded above by some constant multiple of $g(n)$ for all large n , i.e., if there exist some positive constant c and some nonnegative integer n_0 such that

$$t(n) \leq cg(n) \text{ for all } n \geq n_0.$$

- E.g., $100n + 5 \in O(n^2)$

$$100n + 5 \leq 100n + n \text{ (for all } n \geq 5) = 101n \leq 101n^2$$

$$\text{i.e., } c = 101, n_0 = 5$$



CIS KMUTNB

Discrete mathematics

13

Big-omega

$$t(n) \in \Omega(g(n))$$

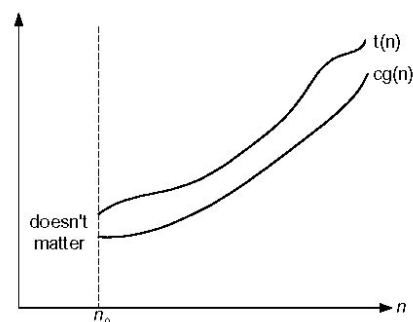
- if $t(n)$ is bounded below by some positive constant multiple of $g(n)$ for all large n , i.e., if there exist some positive constant and some nonnegative integer n_0 such that

$$t(n) \geq cg(n) \text{ for all } n \geq n_0.$$

- E.g., $n^3 \in \Omega(n^2)$

$$n^3 \geq n^2 \text{ for } n \geq 0$$

$$\text{i.e., } c = 1, n_0 = 0$$



CIS KMUTNB

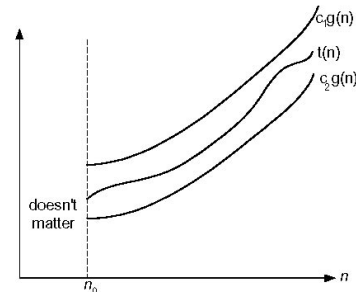
Discrete mathematics

14

Big-theta

- if $t(n)$ is bounded both above and below by some positive constant multiples of $g(n)$ for all large n , i.e., if there exist some positive constant c_1 and c_2 and some nonnegative integer n_0 such that

$$c_2 g(n) \leq t(n) \leq c_1 g(n) \quad \text{for all } n \geq n_0.$$



- E.g., $\frac{1}{2}n(n-1) \in \Theta(n^2)$

$$\mapsto \frac{1}{2}n(n-1) = \frac{1}{2}n^2 - \frac{1}{2}n \leq \frac{1}{2}n^2$$

$$\mapsto \frac{1}{2}n(n-1) = \frac{1}{2}n^2 - \frac{1}{2}n \geq \frac{1}{2}n^2 - \frac{1}{2}n \cdot \frac{1}{2}n = \frac{1}{4}n^2$$

$$c_2 = \frac{1}{4}, c_1 = \frac{1}{2}, n_0 = 2$$

CIS KMUTNB

Discrete mathematics

15

Basic Asymptotic Efficiency classes

1	constant
$\log n$	logarithmic
n	linear
$n \log n$	$n \log n$
n^2	quadratic
n^3	cubic
2^n	exponential
$n!$	factorial

CIS KMUTNB

Discrete mathematics

16

การใช้ทฤษฎีลิมิตเพื่อเปรียบเทียบอัตราการเติบโต

$$\lim_{n \rightarrow \infty} \frac{t(n)}{g(n)} = \begin{cases} 0 & \text{อัตราการเติบโตของ } t(n) \text{ น้อยกว่า } g(n) \\ c > 0 & \text{อัตราการเติบโตของ } t(n) \text{ เท่ากับ } g(n) \\ \infty & \text{อัตราการเติบโตของ } t(n) \text{ มากกว่า } g(n) \end{cases}$$

Examples:

$$- 10n \quad \text{vs.} \quad 2n^2$$

$$- n(n+1)/2 \quad \text{vs.} \quad n^2$$

L'Hôpital's rule

If

$$\lim_{n \rightarrow \infty} f(n) = \lim_{n \rightarrow \infty} g(n) = \infty$$

The derivatives f' , g' exist,

Then

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{f'(n)}{g'(n)}$$

Example: $\log n$ vs. n

Mathematical analysis of **non-recursive** algorithms

- Non-recursive Algorithm Time Efficiency
- ระบุขนาดของข้อมูล (*input size*) และ basic operation ของอัลกอริทึม
 - Basic operation มักที่อยู่ในรูปขั้นในสุด
- พิจารณาว่าอัลกอริทึมมี *worst*, *average*, and *best* case หรือไม่
 - จำนวนครั้งการทำงานของ basic operation แตกต่างกันเมื่อข้อมูลต่างกัน แม้ขนาดข้อมูลเท่ากัน
- ตั้งสมการ summation ของ basic operation
- แก้สมการเพื่อหา Order of growth

อัลกอริทึม Selection sort

Algorithm SelectionSort($A[0..n-1]$)

```

for i ← 0 to n-2 do
  min ← i
  for j ← i+1 to n-1 do
    if  $A[j] < A[min]$ 
      min ← j
  swap  $A[i]$  and  $A[min]$ 
  
```

Time Efficiency?

อัลกอริทึม Insertion sort

Algorithm InsertionSort($A[0..n-1]$)

```

for i ← 1 to n-1 do
  v ← A[ i ]
  j ← i-1
  while j ≥ 0 and A[ j ] > v do
    A[ j+1 ] ← A[ j ]
    j ← j-1
  A[ j+1 ] ← v

```

Time Efficiency?

CIS KMUTNB

Discrete mathematics

21

Mathematical analysis of recursive algorithms

- Recursive Algorithm Time Efficiency
- ระบุขนาดของข้อมูล (*input size*) และ basic operation ของอัลกอริทึม
 - Basic operation มักที่อยู่ในรูปขั้นในสุด
- พิจารณาว่าอัลกอริทึมมี *worst*, *average*, and *best* case หรือไม่
 - จำนวนครั้งการทำงานของ basic operation แตกต่างกันเมื่อข้อมูลต่างกัน แม้ขนาดข้อมูลเท่ากัน
- ตั้งสมการ Recurrence relation ของ basic operation
 - ต้องระบุ initial condition ด้วยเสมอ
- แก้สมการเพื่อหา Order of growth

CIS KMUTNB

Discrete mathematics

22

Tower of Hanoi

- การย้าย N disks จากเสา 1 ไปยังเสา 3 โดยมีเสา 2 เป็นเสาพัก ทำได้โดย

1. ย้าย (N-1) disks จากเสา 1 ไปยังเสา 2
2. ย้าย disk แผ่นล่างสุด ไปยังเสา 3
3. ย้าย (N-1) disks จากเสา 2 ไปยังเสา 3

- จำนวนครั้งของการเคลื่อนย้าย

$$H(n) = H(n-1) + 1 + H(n-1), \text{ for } n > 1$$

$$\text{และ } H(1) = 1$$

Solve The Tower of Hanoi recurrence relation.

$$\begin{aligned}
 H_n &= 2H_{n-1} + 1 \\
 &= 2(2H_{n-2} + 1) + 1 = 2^2H_{n-2} + 2 + 1 \\
 &= 2^2(2H_{n-3} + 1) + 2 + 1 = 2^3H_{n-3} + 2^2 + 2 + 1 \\
 &\vdots \\
 &= 2^{n-1}H_1 + 2^{n-2} + 2^{n-3} + \dots + 2 + 1 \\
 &= 2^{n-1} + 2^{n-2} + \dots + 2 + 1 \\
 &= 2^n - 1.
 \end{aligned}$$

Important recurrence types:

- One (constant) operation reduces problem size by one.

$$T(n) = T(n-1) + c \quad T(1) = d$$

$$\text{Solution: } T(n) = (n-1)c + d \quad \text{linear}$$

- A pass through input reduces problem size by one.

$$T(n) = T(n-1) + cn \quad T(1) = d$$

$$\text{Solution: } T(n) = [n(n+1)/2 - 1]c + d \quad \text{quadratic}$$

- One (constant) operation reduces problem size by half.

$$T(n) = T(n/2) + c \quad T(1) = d$$

$$\text{Solution: } T(n) = c \lg n + d \quad \text{logarithmic}$$

- A pass through input reduces problem size by half.

$$T(n) = 2T(n/2) + cn \quad T(1) = d$$

$$\text{Solution: } T(n) = cn \lg n + d n \quad n \log n$$

แบบฝึกหัด 2

- ข้อใดผิด

Big-Oh Examples

$$\begin{aligned} n &\in O(n) ? \\ 10n &\in O(n) ? \\ n &\in O(10n) ? \\ n &\in O(n^2) ? \\ n^2 &\in O(n) ? \\ 10n^2 &\in O(n^2) ? \\ n \lg n &\in O(n^2) ? \\ \ln n &\in O(2n) ? \\ \lg n &\in O(n) ? \\ 3n + 4 &\in O(n) ? \\ 5n^2 + 10n - 2 &\in O(n^3) ? O(n^2) ? O(n) ? \end{aligned}$$

Big-Oh Examples

$$\begin{aligned} \sqrt{n} &\in O(n) ? \\ \lg n &\in O(2^n) ? \\ \lg n &\in O(n) ? \\ n \lg n &\in O(n) ? \\ n \lg n &\in O(n^2) ? \\ \sqrt{n} &\in O(\lg n) ? \\ \lg n &\in O(\sqrt{n}) ? \\ n \lg n &\in O(n^{\frac{3}{2}}) ? \\ n^3 + n \lg n + n\sqrt{n} &\in O(n \lg n) ? \\ n^3 + n \lg n + n\sqrt{n} &\in O(n^3) ? \\ n^3 + n \lg n + n\sqrt{n} &\in O(n^4) ? \end{aligned}$$

แบบฝึกหัด 3

- Calculating Big-O ของ code ต่อไปนี้

1.

 n

```
for (i=1; i<n; i=i+2)
    sum++;
```

2.

 n^2

```
for (i=1; i<n; i++)
    for (j=1; j < n/2; j++)
        sum++;
```

```
for (i=0; i<n; i++)
    for (j = 0; j<i; j++)
        sum++;
```

3.

 $n^2 \lg n$

```
for (i=0; i<n; i++)
    for (j=1; j<n; j=j*2)
        for (k=1; k<n; k=k+2)
            sum++;
```

แบบฝึกหัด 4

- (Program) จงเขียนโปรแกรมคูณเลขฐาน 2 $(10101011101010101)_2 \times (10101011101010101)_2$ พร้อมทั้งคำนวณ Big-O โดยให้คุณที่ละหลักจากหลังไปหน้า
- (Program) จงเขียนโปรแกรมเรียงข้อมูลของ $A[n]$ จากน้อยไปมาก โดย Input คือ จำนวนข้อมูลและข้อมูล โดยเวลาที่ใช้ในการประมวลผลคือ $O(n \log n)$
- (Program) จงเขียนโปรแกรมเรียงข้อมูลของ $A[n]$ จากน้อยไปมาก โดย Input คือ จำนวนข้อมูลและข้อมูล โดยเวลาที่ใช้ในการประมวลผลคือ $O(n)$

แบบฝึกหัด 5

(Program) กำหนดให้เมทริกซ์

A =	2	5	6	3	7
	6	2	8	7	9
	3	1	4	7	8
	3	9	7	8	2
	3	5	6	7	2

B =	-1	5	0	6	7
	6	5	1	4	7
	2	5	8	10	9
	9	6	3	2	5
	1	7	2	5	3

จงเขียนโปรแกรมหาผลลัพธ์ $C = A \times B$ พร้อมทั้งคำนวณ Big-O

Divide and Conquer

- General idea:**

Divide a problem into subprograms of the same kind; solve subprograms using the same approach, and combine partial solution (if necessary).



1. Find the maximum and minimum

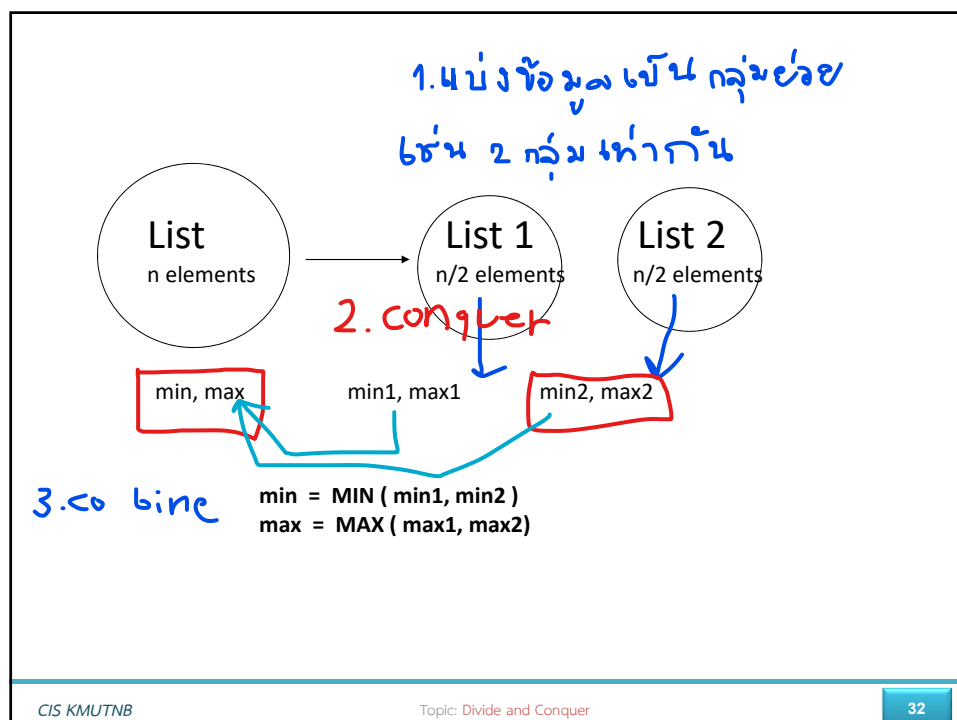
The problem: Given a list of unordered n elements, find max and min

The straightforward algorithm:

```

max  $\leftarrow$  min  $\leftarrow$  A (1);
for  $i \leftarrow 2$  to  $n$  do
    if A ( $i$ ) > max, max  $\leftarrow$  A ( $i$ );
    if A ( $i$ ) < min, min  $\leftarrow$  A ( $i$ );
  
```

Key comparisons: $2(n - 1)$



The Divide-and-Conquer algorithm:

```

procedure Rmaxmin (i, j, fmax, fmin); // i, j are index #, fmax,
begin // fmin are output parameters
  case:
    i = j:      fmax ← fmin ← A (i);
    i = j - 1:  if A (i) < A (j) then { fmax ← A (j);
                                      fmin ← A (i);
                                else { fmax ← A (i);
                                      fmin ← A (j);

    else:      mid ← ⌊  $\frac{i+j}{2}$  ⌋;
               call Rmaxmin (i, mid, gmax, gmin);
               call Rmaxmin (mid+1, j, hmax, hmin);
               fmax ← MAX (gmax, hmax);
               fmin ← MIN (gmin, hmin);

  end
end;

```

CIS KMUTNB

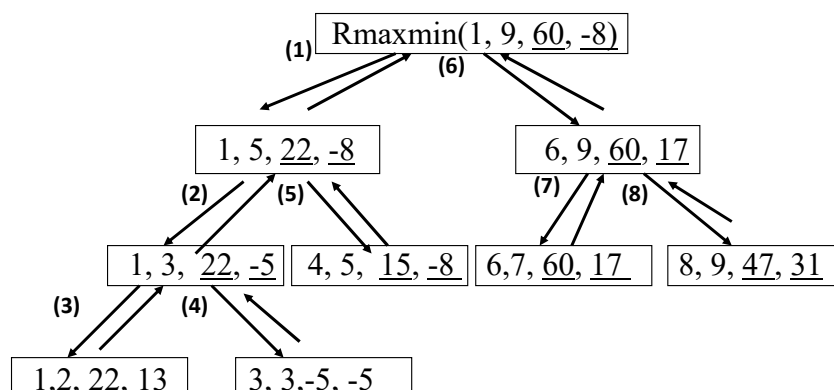
Topic: Divide and Conquer

33

Eg: find max and min in the array:

22, 13, -5, -8, 15, 60, 17, 31, 47 (n = 9)

Index:	1	2	3	4	5	6	7	8	9
Array:	22	13	-5	-8	15	60	17	31	47



CIS KMUTNB

Topic: Divide and Conquer

34

Analysis: For algorithm containing recursive calls, we can use recurrence relation to find its complexity

$T(n)$ - # of comparisons needed for Rmaxmin

Recurrence relation:

$$\begin{cases}
 T(n) = 0 & n = 1 \\
 T(n) = 1 & n = 2 \\
 T(n) = 2T\left(\frac{n}{2}\right) + 2 & \text{otherwise}
 \end{cases}$$

$T(n) = 2T\left(\frac{n}{2}\right) + 2$

$= 2 \cdot (2T\left(\frac{n}{4}\right) + 2) + 2 = 2^2 \cdot T\left(\frac{n}{2^2}\right) + 2^2 + 2$

$= 2^2 \cdot (2T\left(\frac{n}{8}\right) + 2) + 2^2 + 2 = 2^3 \cdot T\left(\frac{n}{2^3}\right) + 2^3 + 2^2 + 2$

...

Assume $n = 2^k$ for some integer k

$$\begin{aligned}
 &= 2^{k-1} T\left(\frac{n}{2^{k-1}}\right) + (2^{k-1} + 2^{k-2} + \dots + 2^1) \\
 &= 2^{k-1} \cdot T(2) + (2^k - 2) = \frac{n}{2} \cdot 1 + n - 2 \\
 &= 1.5n - 2
 \end{aligned}$$

They don't have to be the same constant.
We make them the same here for simplicity.
It will not affect the overall result.

Theorem:

Claim: $T(n) = \begin{cases} b & n = 1 \\ aT(\frac{n}{c}) + bn & n > 1 \end{cases}$ where are a, b, c constants

Handwritten notes:
 $n=1$ (blue arrow from b)
 $n > 1$ (blue arrow from $aT(\frac{n}{c}) + bn$)
 $aT(\frac{n}{c})$ (blue arrow from a)
 bn (blue arrow from b)
 $reservoir$ (blue arrow from $aT(\frac{n}{c})$)
 $combine$ (blue arrow from bn)

$T(n) = \begin{cases} O(n) & a < c \text{ (blue note: ฐานคูณน้อยกว่าค่าที่ใส่)} \\ O(n \log n) & a = c \text{ (blue note: ฐานคูณเท่า)} \\ O(n^{\log_c a}) & a > c \text{ (blue note: ฐานคูณมากกว่า)} \end{cases}$

CIS KMUTNB Topic: Divide and Conquer 37

Proof: Assume $n = c^k$

$$T(n) = a \cdot T(\frac{n}{c}) + bn$$

$$T(n) = a \cdot (a \cdot T(\frac{n}{c^2}) + b \cdot \frac{n}{c}) + bn = a^2 \cdot T(\frac{n}{c^2}) + ab \cdot \frac{n}{c} + bn$$

$$= a^2 \cdot (a \cdot T(\frac{n}{c^3}) + b \cdot \frac{n}{c^2}) + ab \cdot \frac{n}{c} + bn$$

$$= a^3 \cdot T(\frac{n}{c^3}) + bn \cdot (\frac{a^2}{c^2} + \frac{a}{c} + 1)$$

$$\dots$$

$$= a^k \cdot T(\frac{n}{c^k}) + bn \cdot (\frac{a^{k-1}}{c^{k-1}} + \frac{a^{k-2}}{c^{k-2}} + \dots + \frac{a^0}{c^0})$$

$$= a^k \cdot b + bn \cdot ((\frac{a}{c})^{k-1} + (\frac{a}{c})^{k-2} + \dots + (\frac{a}{c})^0)$$

$$= a^k \cdot b \cdot \frac{n}{c^k} + bn \cdot ((\frac{a}{c})^{k-1} + (\frac{a}{c})^{k-2} + \dots + (\frac{a}{c})^0)$$

$$= bn \cdot (\frac{a}{c})^k + bn \cdot ((\frac{a}{c})^{k-1} + (\frac{a}{c})^{k-2} + \dots + (\frac{a}{c})^0)$$

$$= bn \cdot (\sum_{i=0}^k (\frac{a}{c})^i)$$

CIS KMUTNB Topic: Divide and Conquer 38

If $a < c$, $\sum_{i=0}^k \left(\frac{a}{c}\right)^i$ is a constant

$$T(n) = bn \cdot \text{a constant}$$

$$\therefore T(n) = O(n)$$

if $a = c$, $\sum_{i=0}^k \left(\frac{a}{c}\right)^i = k + 1$

$$T(n) = bn \sum_{i=0}^k 1 = bn \cdot (k + 1)$$

$$= bn \cdot (\log_c n + 1)$$

$$= O(n \log n)$$

$$\text{If } a > c, \sum_{i=0}^k \left(\frac{a}{c}\right)^i = \frac{\left(\frac{a}{c}\right)^{k+1} - 1}{\left(\frac{a}{c}\right) - 1}$$

$$T(n) = bn \cdot \sum_{i=0}^k \left(\frac{a}{c}\right)^i = bn \cdot \frac{\left(\frac{a}{c}\right)^{k+1} - 1}{\left(\frac{a}{c}\right) - 1}$$

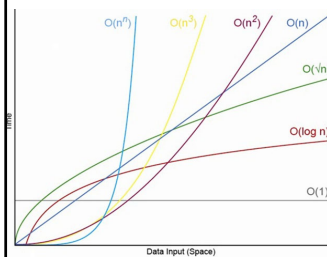
$$\leq bn \cdot \left(\frac{a}{c}\right)^k = b \cdot a^k = b \cdot a^{\log_c n}$$

$$= b \cdot n^{\log_c a}$$

$$= O(n^{\log_c a})$$

Divide and Conquer

- Binary search



ALGORITHM 6 A Recursive Binary Search Algorithm.

```

procedure binary search(i, j, x: i, j, x integers,  $1 \leq i \leq j \leq n$ )
   $m := \lfloor (i + j) / 2 \rfloor$ 
  if  $x = a_m$  then
    return  $m$ 
  else if ( $x < a_m$  and  $i < m$ ) then
    return binary search( $i, m - 1, x$ )
  else if ( $x > a_m$  and  $j > m$ ) then
    return binary search( $m + 1, j, x$ )
  else return 0
  {output is location of  $x$  in  $a_1, a_2, \dots, a_n$  if it appears; otherwise it is 0}
  
```

- Brute force solution: $O(n)$
- Divide and Conquer solution: $O(\log n)$

CIS KMUTNB

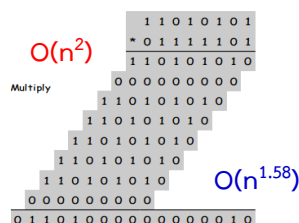
Divide and Conquer

41

Divide and Conquer Concept

- Break up problem into several parts
- Solve each part recursively
- Combine solutions to sub-problems into overall solution
- Examples

Integer Multiplication



Matrix Multiplication

$O(n^3)$

$$C_{ij} = \sum_{k=1}^N a_{ik} b_{kj}$$

$$\begin{pmatrix} 26 & 62 & 98 \\ 80 & 224 & 368 \\ 134 & 386 & 638 \end{pmatrix} = \begin{pmatrix} 0 & 2 & 4 \\ 6 & 8 & 10 \\ 12 & 14 & 16 \end{pmatrix} \times \begin{pmatrix} 1 & 7 & 13 \\ 3 & 9 & 15 \\ 5 & 11 & 17 \end{pmatrix}$$

$O(n^{2.81})$

$$\begin{pmatrix} c_{11} & c_{12} & c_{13} & \dots & c_{1N} \\ c_{21} & c_{22} & c_{23} & \dots & c_{2N} \\ c_{31} & c_{32} & c_{33} & \dots & c_{3N} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ c_{N1} & c_{N2} & c_{N3} & \dots & c_{NN} \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1N} \\ a_{21} & a_{22} & a_{23} & \dots & a_{2N} \\ a_{31} & a_{32} & a_{33} & \dots & a_{3N} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{N1} & a_{N2} & a_{N3} & \dots & a_{NN} \end{pmatrix} \times \begin{pmatrix} b_{11} & b_{12} & b_{13} & \dots & b_{1N} \\ b_{21} & b_{22} & b_{23} & \dots & b_{2N} \\ b_{31} & b_{32} & b_{33} & \dots & b_{3N} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ b_{N1} & b_{N2} & b_{N3} & \dots & b_{NN} \end{pmatrix}$$

CIS KMUTNB

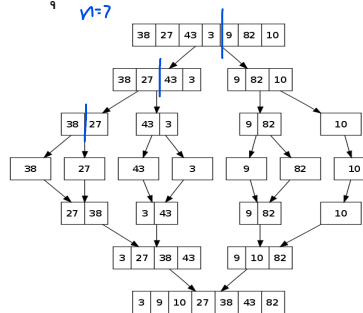
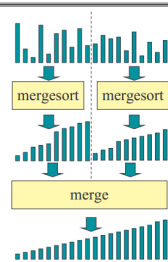
Divide and Conquer

42

Merge Sort

- การเรียงลำดับแบบผสาน โดยการแบ่งแถวลำดับออกเป็น 2 ชุดย่อย ชุดทางซ้ายและชุดทางขวาวขนาดพอๆ กัน
- นำข้อมูลทั้ง 2 ชุดมาเรียงลำดับ
- นำผลของสิ่งที่ได้มาผสานกัน ผลลัพธ์สุดท้ายจะเป็นผลลัพธ์ที่เรียงลำดับ

Merge Sort : Divide & Conquer



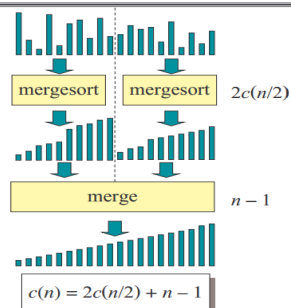
CIS KMUTNB

Divide and Conquer

43

Merge sort analysis

Merge Sort : Analysis



Merge Sort : Analysis

$$\begin{aligned}
 c(n) &= 2c(n/2) + (n-1) \quad n > 1, c(0) = c(1) = 0, \quad n = 2^k \\
 &= 2(2c(n/2^2) + (n/2 - 1)) + (n-1) \\
 &= 2^2 c(n/2^2) + (n-2) + (n-1) \\
 &= 2^3 c(n/2^3) + (n-4) + (n-2) + (n-1) \\
 &\dots \\
 &= 2^k c(n/2^k) + kn - (2^{k-1} + 2^{k-2} + \dots + 2^0) \\
 &= n \log n - n + 1 \\
 &= O(n \log n)
 \end{aligned}$$

CIS KMUTNB

Divide and Conquer

44

3. Merge Sort

The problem:

Given a list of n numbers, sort them in non-decreasing order

idea: Split each part into 2 equal parts and sort each part using Mergesort, then merge the two sorted sub-lists into one sorted list.

The algorithm:

```

procedure MergeSort ( $low, high$ )
  begin     $low < high$ 
    if
    then
       $mid \leftarrow \left\lfloor \frac{low + high}{2} \right\rfloor$ 
      call MergeSort ( $low, mid$ );
      call MergeSort ( $mid+1, high$ );
      call Merge ( $low, mid, high$ );
    end;
  
```

CIS KMUTNB

Topic: Divide and Conquer

45

```

procedure Merge ( $low, mid, high$ )
  
```

```

  begin     $i \leftarrow low, j \leftarrow mid + 1, k \leftarrow low$ 
  
```

```

    while ( $i \leq mid$  and  $j \leq high$ )
  
```

```

      if  $A(i) < A(j)$ , then  $\left[ \begin{array}{l} U(k) \leftarrow A(i); \\ i++; \end{array} \right.$ 
      else  $\left[ \begin{array}{l} U(k) \leftarrow A(j); \\ j++; \end{array} \right.$ 
  
```

```

       $k++$ ;
  
```

```

    if ( $i > mid$ )
  
```

```

      move  $A(j)$  through  $A(high)$  to  $U(k)$  through  $U(high)$ ;
  
```

```

    else
  
```

```

      move  $A(i)$  through  $A(mid)$  to  $U(k)$  through  $U(high)$ ;
  
```

```

  end;
  
```

CIS KMUTNB

Topic: Divide and Conquer

46

Analysis:

Worst-Case for MergeSort is:

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + bn$$
$$= O(n \log n)$$

Average-Case? Merge sort pays no attention to the original order of the list, it will keep divide the list into half until sub-lists of length 1, then start merging.

Therefore Average-Case is the same as the Worst-Case.

4. Quick Sort

The problem:

Same as Merge Sort

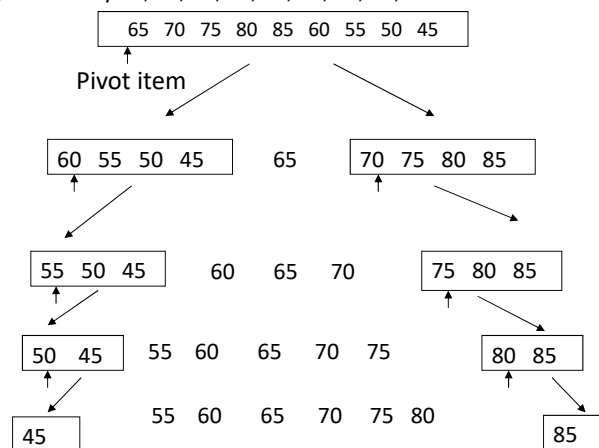
Compared with Merge Sort:

- Quick Sort also use Divide-and-Conquer strategy
- Quick Sort eliminates merging operations

How it works?

Use partition

Eg: Sort array 65, 70, 75, 80, 85, 60, 55, 50, 45



CIS KMUTNB

Topic: Divide and Conquer

49

The algorithm:

```

procedure QuickSort ( $p, q$ )
  begin
    if  $p < q$ ,
    then { call Partition ( $p, q, pivotposition$ );
          call QuickSort ( $p, pivotposition - 1$ );
          call QuickSort ( $pivotposition + 1, q$ );
        }
    end;

procedure Partition ( $low, high, pivotposition$ )
  begin
     $v \leftarrow A(low)$ ;
     $j \leftarrow low$ ;
    for  $i \leftarrow (low + 1)$  to  $high$ 
      if  $A(i) < v$ ,
      {  $j++$ ; swap
         $A(i) \leftrightarrow A(j)$ ;
      }
     $pivotposition \leftarrow j$ ;
     $A(low) \leftrightarrow A(pivotposition)$ ;
  end;
  
```

CIS KMUTNB

Topic: Divide and Conquer

50

*Analysis:***Worst-Case:**

Call Partition $O(n)$ times, each time takes $O(n)$ steps. So $O(n^2)$, and it is worse than Merge Sort in the Worst-Case.

Best-Case:

Split the list evenly each time.

$$O(n \log n)$$

Average-Case:

Assume pivot is selected randomly, so the probability of pivot being the k^{th} element is equal, $\forall k$.

$$\text{prob}(\text{pivot} \leftarrow k) = \frac{1}{n}, \quad \forall k$$

$C(n)$ = # of key comparison for n items

- average it over all k , ($C_A(n)$ is average performance)

$$= (n-1) + C \cdot (k-1) + C \cdot (n-k)$$

- multiply both side by n

$$C_A(n) = (n-1) + \frac{1}{n} \sum_{k=1}^n (C_A(k-1) + C_A(k-2))$$

$$n \cdot C_A(n) =$$

$$n(n-1) + 2 \cdot (C_A(0) + C_A(1) + \dots + C_A(n-1)) \quad (1)$$

- replace n by $n-1$

$$(n-1) \cdot C_A(n-1) = (n-1)(n-2) + 2 \cdot (C_A(0) + \dots + C_A(n-2)) \quad (2)$$

- subtract (2) from (1)

$$\begin{aligned} n \cdot C_A(n) - (n-1)C_A(n-1) &= 2 \cdot (n-1) + 2 \cdot C_A(n-1) \\ \Rightarrow n \cdot C_A(n) &= 2 \cdot (n-1) + (n+1) \cdot C_A(n-1) \end{aligned}$$

- divide $n(n+1)$ for both sides

$$\begin{aligned} \frac{C_A(n)}{n+1} &= \frac{C_A(n-1)}{n} + \frac{2 \cdot (n-1)}{n(n+1)} \\ &= \frac{C_A(n-2)}{n-1} + \frac{2 \cdot (n-2)}{(n-1) \cdot n} + \frac{2 \cdot (n-1)}{n(n+1)} \\ &= \frac{C_A(n-3)}{n-2} + \frac{2 \cdot (n-3)}{(n-2) \cdot (n-1)} + \frac{2 \cdot (n-2)}{(n-1) \cdot n} + \frac{2 \cdot (n-1)}{n(n+1)} \\ &\quad \dots \end{aligned}$$

$$\left[\begin{array}{l} \text{Note:} \\ C_A(1) = 0 \end{array} \right]$$

$$= \frac{C_A(1)}{2} + 2 \cdot \left(\frac{1}{2 \cdot 3} + \frac{2}{3 \cdot 4} + \dots + \frac{n-1}{n(n+1)} \right)$$

$$= 0 + 2 \cdot \sum_{k=2}^n \frac{k-1}{k(k+1)}$$

$$\begin{aligned} \therefore \text{Best-Case} &= \text{Average-Case} \\ &\leq 2 \cdot \sum_{k=2}^n \frac{1}{k} \leq 2 \cdot \int_1^n \frac{1}{k} dk = 2 \cdot (\log_e n - \log_e 1) \\ &= O(\log n) \\ \Rightarrow C_A(n) &= O(n \log n) \end{aligned}$$

5. Selection

The problem:

Given a list of n elements find the k^{th} smallest one

Note: special cases: when $k = 1$, *min*
when $k = n$, *max*

The solving strategy:

If use sorting strategy like MergeSort, it takes $O(n \log n)$

If use Quick Sort, Best-Case is $O(n)$, Worst-Case will call partition $O(n)$ times, each time takes $O(n)$, so Worst-Case is $O(n^2)$.

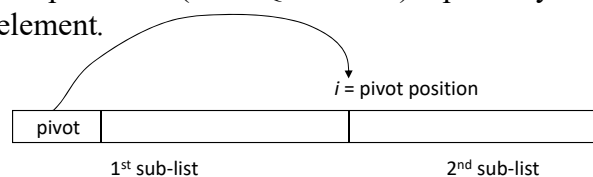
CIS KMUTNB

Topic: Divide and Conquer

55

Select1 – the first algorithm

idea: use “partition” (from Quick Sort) repeatedly until we find the k^{th} element.



For each iteration:

If pivot position = $k \Rightarrow$ Done!

If pivot position < $k \Rightarrow$ to select $(k-i)^{\text{th}}$ element in the 2nd sub-list.

If pivot position > $k \Rightarrow$ to select k^{th} element in the 1st sub-list.

CIS KMUTNB

Topic: Divide and Conquer

56

```

procedure Select1 (A, n, k)  // A is array, n is # of array, k is key
begin
    m ← 1;
    j ← n;
    loop
        call Partition (m, j, pivotposition);
        case:
            k = pivotposition:
                return A(k);
            k < pivotposition:
                j ← pivotposition - 1;
            else:
                m ← pivotposition + 1;
        end loop;
    end;

```

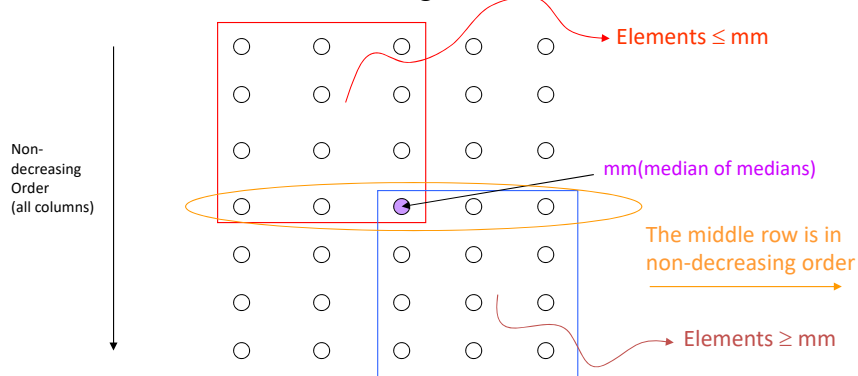
CIS KMUTNB

Topic: Divide and Conquer

57

- 3) By choosing pivot more carefully, we can obtain a selection algorithm with Worst-Case complexity $O(n)$

How: Make sure pivot v is chosen s.t. at least some fraction of the elements will be smaller than v and at least some other fraction of elements will be greater than v .



CIS KMUTNB

Topic: Divide and Conquer

58

- 3) By choosing pivot more carefully, we can obtain a selection algorithm with Worst-Case complexity $O(n)$

Use *mm* (median of medians) rule to choose pivot

procedure Select2 (A, n, k)

begin

if $n \leq r$, then sort A and return the k^{th} element;

divide A into $\left\lfloor \frac{n}{r} \right\rfloor$ subset of size r each, ignore excess

elements, and let $M = \left\{ m_1, m_2, \dots, m_{\left\lfloor \frac{n}{r} \right\rfloor} \right\}$ be the set of

medians of the $\left\lfloor \frac{n}{r} \right\rfloor$ subsets;

CIS KMUTNB

Topic: Divide and Conquer

59

$v \leftarrow \text{Select2} \left(M, \left\lfloor \frac{n}{r} \right\rfloor, \left\lceil \frac{\left\lfloor \frac{n}{r} \right\rfloor}{2} \right\rceil \right);$

use “Partition” to partition A using v as the pivot;

// Assume v is at *pivotposition*

case:

$k = \text{pivotposition}$: return (v);

$k < \text{pivotposition}$: let S be the set of elements
 $A(1, \dots, \text{pivotposition} - 1)$,
 return Select2 ($S, \text{pivotposition} - 1, k$);

else: let R be the set of element
 $A(\text{pivotposition} + 1, \dots, n)$,
 return Select2 ($R, n - \text{pivotposition}, k - \text{pivotposition}$);

end case;

end;

CIS KMUTNB

Topic: Divide and Conquer

60

Analysis:

- How many M_i 's $\leq$ or $\geq mm$?

At least $\left\lceil \left\lfloor \frac{n}{r} \right\rfloor / 2 \right\rceil$

- How many elements $\leq$ or $\geq mm$?

At least $\left\lceil \frac{r}{2} \cdot \left\lfloor \frac{n}{r} \right\rfloor / 2 \right\rceil$

- How many elements in $|R| > mm$ or $|S| < mm$?

At most $n - \left(\left\lceil \frac{r}{2} \right\rceil \cdot \left\lfloor \frac{n}{r} \right\rfloor / 2 \right)$

Assume $r = 5$ $\therefore = n - \left(3 \cdot \left\lceil \frac{n}{5} \right\rceil / 2 \right) \leq n - 1.5 \left\lfloor \frac{n}{5} \right\rfloor$

$$\leq n - 1.5 \cdot \frac{n-4}{5} = 0.7n + 1.2$$

$$\leq 0.75n \quad (n \geq 24)$$

CIS KMUTNB

Topic: Divide and Conquer

61

- Therefore, procedure Select2 the Worst-Case complexity is:

$$T(n) = T\left(\frac{n}{5}\right) + T\left(\frac{3}{4}n\right) + cn$$

where c is chosen sufficiently large, such that
 $T(n) \leq cn$ for $n < 24$.

Proof:

Use induction to show $T(n) \leq 20 \cdot cn$ ($n \geq 24$)

IB (induction base):

$$n = 24, \quad T(n) = T\left(\frac{24}{5}\right) + T\left(\frac{3}{4} \cdot 24\right) + c \cdot 24$$

$$\leq cn + cn + 24c \leq 20 \cdot cn$$

CIS KMUTNB

Topic: Divide and Conquer

62

IH (induction hypothesis):

Suppose $T(n) \leq 20 \cdot cn \quad \forall 24 \leq n < m$

IS (induction solution):

When $n = m$;

$$\begin{aligned} T(m) &= T\left(\frac{m}{5}\right) + T\left(\frac{3}{4}m\right) + cm \\ &\leq 20 \cdot c \cdot \frac{m}{5} + 20 \cdot c \cdot \frac{3}{4} \cdot m + cm \\ &\leq 20 \cdot cn \end{aligned}$$

$\therefore T(n) = O(n)$ complexity of Select2 of n elements

ตัวอย่างของปัญหาที่เหมาะสมสำหรับอัลกอริทึม แบ่งแยกและเอาชนะ

- การคูณเลขจำนวนเต็ม (integer multiplication)
- การคูณเมตริกซ์ (matrix multiplication)
- เซตของจุดที่ครอบคลุม (maxima set of points)

2. Integer multiplication

The problem:

Multiply two large integers (n digits)

The traditional way:

Use two for loops, it takes operations

Integer multiplication with Brute force

ในการคำนวณตัวเลข การคูณ (multiplication) และการหาร (division) ถือเป็นงานที่ต้องใช้ทรัพยากรในการคำนวณสูงเมื่อเปรียบเทียบกับ การบวก (addition) และการลบ (subtraction) โดยเฉพาะอย่างยิ่งหากตัวเลขมีจำนวนหลายๆ หลัก

วิธีการคูณเลข 2 จำนวน ที่เรารู้กันเคยมากที่สุด คือ *pen-and-pencil algorithm* โดยเริ่มจากการพิจารณาคำคูณในแต่ละหลักไปทุกหลักของตัวตั้ง จากนั้นเลื่อนตัวคูณไปหลักถัดไป และทำซ้ำจนกระทั่งครบทุกหลัก และนำมาผลลัพธ์บวกกัน

	23958233
×	5830
	00000000
	71874699
	191665864
+	119791165
	139676498390

จากตัวอย่างจะเห็นได้ว่า หากตัวตั้งและตัวคูณมีจำนวนหลักเท่ากันคือ n

จำนวนครั้งในการคูณทั้งหมดคือ n^2

Integer multiplication with divide & conquer

เพื่อลดจำนวนครั้งในการคูณเราอาจใช้เทคนิคการแบ่งแยกและเอาชนะสำหรับแก้ปัญหานี้ได้ แต่เราจะต้องหาวิธีในการแบ่งปัญหาและการรวมปัญหาให้ได้ ซึ่งในปัญหานี้อาจกระทำได้ง่าย ดังนี้

สมมติให้เลขจำนวนเต็ม 2 หลักคือ 23 และ 14 เราสามารถเขียนในอีกรูปแบบดังนี้

$$23 = 2 \cdot 10^1 + 3 \cdot 10^0$$

$$14 = 1 \cdot 10^1 + 4 \cdot 10^0$$

การแบ่งปัญหาย่อย



$$23 * 14 = (2 \cdot 10^1 + 3 \cdot 10^0) * (1 \cdot 10^1 + 4 \cdot 10^0) \quad \text{การรวมคำตอบ}$$

$$23 * 14 = (2 * 1)10^2 + (2 * 4 + 3 * 1)10^1 + (3 * 4)10^0 = 322$$

Integer multiplication with divide & conquer

ตั้งนั้นเลขจำนวนเต็ม 2 ตัว คือ X และ Y ขนาด n หลัก สามารถแบ่งปัญหาย่อยได้ครั้งละ 4 เทอม คือ a, b, c และ d สำหรับตัวตั้งและตัวคูณ ดังนี้

$$x \begin{bmatrix} a & b \end{bmatrix}$$

$$x = a \cdot 10^{n/2} + b$$

$$y \begin{bmatrix} c & d \end{bmatrix}$$

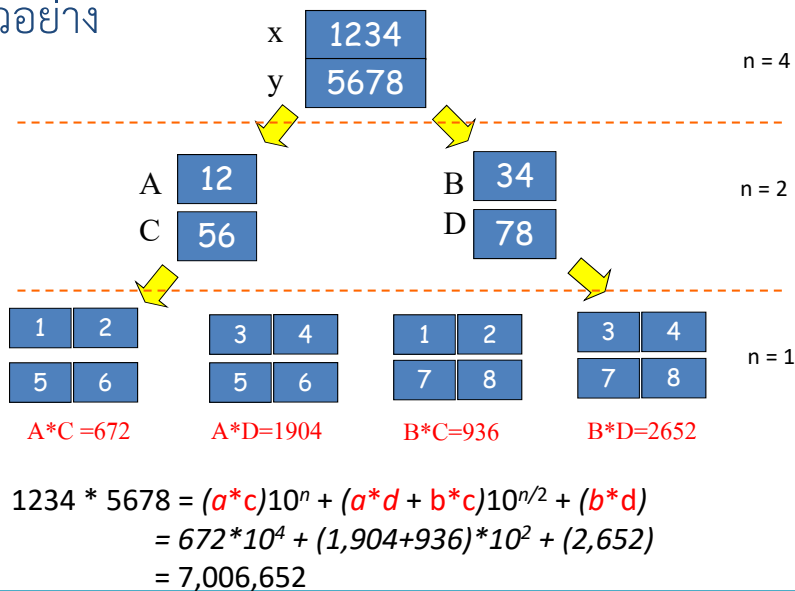
$$y = c \cdot 10^{n/2} + d$$

และการรวมคำตอบของปัญหาย่อยคือ

$$x * y = (a \cdot 10^{n/2} + b) * (c \cdot 10^{n/2} + d)$$

$$x * y = (a * c) \cdot 10^n + (a * d + b * c) \cdot 10^{n/2} + b * d$$

ตัวอย่าง



CIS KMUTNB

Divide-and-Conquer

69

The Divide-and-Conquer way:

Suppose large integers, , divide into two part a and b , same as into c and d .

$$x: \begin{bmatrix} a & b \end{bmatrix} \quad \therefore \quad x = a \cdot 10^{\frac{n}{2}} + b$$

$$y: \begin{bmatrix} c & d \end{bmatrix} \quad \therefore \quad y = c \cdot 10^{\frac{n}{2}} + d$$

$$x \cdot y = (a \cdot 10^{\frac{n}{2}} + b)(c \cdot 10^{\frac{n}{2}} + d)$$

$$= \underline{ac} \cdot 10^n + \underline{bd} + 10^{\frac{n}{2}}(\underline{ad} + \underline{bc})$$

So, transform the problem of multiply two integers of n -digit into four subproblems of multiply two integers of $\frac{n}{2}$ -digit

 $\frac{n}{2}$

CIS KMUTNB

Topic: Divide and Conquer

70

Worst-Case is: $T(n) = 4 \cdot T\left(\frac{n}{2}\right) + bn$

$$= O(n^{\log_2 4}) = O(n^2)$$

however, it is same as the traditional way.

Therefore, we need to improve equation show before:

...

$$= ac \cdot 10^n + bd + 10^{\frac{n}{2}} \cdot (ad + bc)$$

$$= \underline{ac} \cdot 10^n + \underline{bd} + 10^{\frac{n}{2}} \cdot ((\underline{a+b})(\underline{c+d}) - ac - bd)$$

Worst-Case is:

$$T(n) = 3 \cdot T\left(\frac{n}{2}\right) + bn$$

$$= O(n^{\log_2 3}) \approx O(n^{1.58})$$

Karatsuba method (1962)

จากหลักการ divide & conquer จะเห็นว่าจำนวนครั้งในการคูณไม่ได้ลดลงไปจากวิธีการ brute force คือ n^2

ต่อมาในปี ค.ศ. 1962 Anatoly Karatsuba จะได้นำเสนอการแยกตัวประกอบในเทอมกลางเพื่อลดจำนวนครั้งในการคูณดังนี้

$$x * y = (a * c).10^n + \boxed{(a * d + b * c).10^{n/2}} + b * d$$

โดยพยายามแยกตัวประกอบในเทอมกลางเพื่อให้เกิดเทอมที่ซ้ำกันออกมา

$$x * y = (a * c).10^n + \boxed{((a + b) * (c + d) - a * c - b * d).10^{n/2}} + b * d$$

จะเห็นว่าด้วยวิธีการแยกตัวประกอบนี้ จะทำให้สามารถลดจำนวนครั้งในการคูณกัน จาก 4 ครั้งในแต่ละรอบของการแบ่งเหลือเพียง 3 ครั้งเท่านั้น

The algorithm by Divide-and-Conquer:

Large-int function multiplication (x,y)

begin

$n = \text{MAX} (\text{\# of digits in } x, \text{\# of digits in } y);$

if (x = 0) or (y = 0), return 0;

else if (n = 1), return $x \cdot y$ in the usual way;

else

$m = \left\lfloor \frac{n}{2} \right\rfloor;$

$a = x \text{ divide } 10^m;$

$b = x \text{ rem } 10^m;$

$c = y \text{ divide } 10^m;$

$d = y \text{ rem } 10^m;$

$p_1 = \text{MULTIPLICATION}(a, c);$

$p_2 = \text{MULTIPLICATION}(b, d);$

$p_3 = \text{MULTIPLICATION}(a+b, c+d);$

return $p_1 \cdot 10^{2m} + p_2 + 10^m(p_3 - p_1 - p_2)$

end;

CIS KMUTNB

Topic: Divide and Conquer

73

example: $x = 143, y = 256$

P	$x = 143$ $y = 256$	$n = 3$	$m = 1$	$a = 14$ $b = 3$ $c = 25$ $d = 6$	$p_1 = p'$ $p_2 = 18$ $p_3 = p''$	$P = 36608$
P'	$x = 14$ $y = 25$	$n = 2$	$m = 1$	$a = 1$ $b = 4$ $c = 2$ $d = 5$	$p_1 = 2$ $p_2 = 20$ $p_3 = 35$	$P' = 350$
P''	$x = 17$ $y = 31$	$n = 2$	$m = 1$	$a = 1$ $b = 7$ $c = 3$ $d = 1$	$p_1 = 3$ $p_2 = 7$ $p_3 = 32$	$P'' = 527$

CIS KMUTNB

Topic: Divide and Conquer

74

CIS KMUTNB
75

6. Matrix multiplication

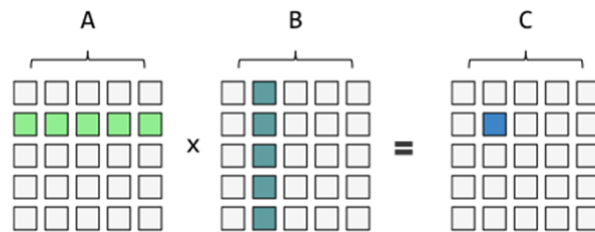
The problem:

Multiply two matrices A and B , each of size $[n \times n]$

$$\begin{bmatrix} A \\ n \times n \end{bmatrix} \cdot \begin{bmatrix} B \\ n \times n \end{bmatrix} = \begin{bmatrix} C \\ n \times n \end{bmatrix}$$

CIS KMUTNB
Topic: Divide and Conquer
76

Matrix multiplication



Brute force algorithm

```
void multiply(int A[][N], int B[][N], int C[][N])
{
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
        {
            C[i][j] = 0;
            for (int k = 0; k <
N; k++)
                C[i][j] +=
A[i][k]*B[k][j];
        }
}
```

CIS KMUTNB

77

The traditional way:

$$C_{ij} = \sum_{k=1}^n A_{ik} \times B_{kj}$$

use three for-loop

$$\therefore T(n) = O(n^3)$$

CIS KMUTNB

Topic: Divide and Conquer

78

Matrix multiplication with divide and conquer

แบ่งเมตริกซ์ A และ B ที่ละ $n/2$ จนกระทั่งเมตริกซ์ A และ B มีขนาด 2×2

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \times \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{bmatrix}$$

A B C

A, B and C are square matrices of size $N \times N$

a, b, c and d are submatrices of A, of size $N/2 \times N/2$

e, f, g and h are submatrices of B, of size $N/2 \times N/2$

หากขนาดของเมตริกซ์ A และ B คือ $n \times n$ จำนวนครั้งในการคูณ สำหรับเมตริกซ์ C คือ n^3

CIS KMUTNB

79

Strassen's Matrix Multiplication (1969)

ลดจำนวนครั้งในการคูณเมตริกซ์ C เหลือเพียง 7 ครั้ง จากปกติ 8 ครั้ง แต่เพิ่มจำนวนครั้งของการบวกและลบ เป็น 18 ครั้ง กรณีที่เมตริกซ์ A และ B มีขนาด 2×2

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \times \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} p5 + p4 - p2 + p6 & p1 + p2 \\ p3 + p4 & p1 + p5 - p3 - p7 \end{bmatrix}$$

A B C

$$\begin{aligned} p1 &= a(f - h) \\ p3 &= (c + d)e \\ p5 &= (a + d)(e + h) \\ p7 &= (a - c)(e + f) \end{aligned}$$

$$\begin{aligned} p2 &= (a + b)h \\ p4 &= d(g - e) \\ p6 &= (b - d)(g + h) \end{aligned}$$

CIS KMUTNB

80

The Divide-and-Conquer way:

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \cdot \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$C_{11} = \underline{A_{11} \cdot B_{11}} + \underline{A_{12} \cdot B_{21}}$$

$$C_{12} = \underline{A_{11} \cdot B_{12}} + \underline{A_{12} \cdot B_{22}}$$

$$C_{21} = \underline{A_{21} \cdot B_{11}} + \underline{A_{22} \cdot B_{21}}$$

$$C_{22} = \underline{A_{21} \cdot B_{12}} + \underline{A_{22} \cdot B_{22}}$$

transform the problem of multiplying A and B , each of size $[n \times n]$ into 8 subproblems, each of size

$$\left[\frac{n}{2} \times \frac{n}{2} \right]$$

$$\therefore T(n) = 8 \cdot T\left(\frac{n}{2}\right) + an^2$$

$$= O(n^3)$$

which an^2 is for addition

so, it is no improvement compared with the traditional way

Eg:

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

use Divide-and-Conquer way to solve it as following:

$$\begin{bmatrix} 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 3 & 3 & 3 & 0 \\ 3 & 3 & 3 & 0 \\ 3 & 3 & 3 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$C_{11} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} + \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 3 & 3 \\ 3 & 3 \end{bmatrix}$$

$$C_{12} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix} + \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 3 & 0 \\ 3 & 0 \end{bmatrix}$$

$$C_{21} = \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} + \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 3 & 3 \\ 0 & 0 \end{bmatrix}$$

$$C_{22} = \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix} + \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 3 & 0 \\ 0 & 0 \end{bmatrix}$$

CIS KMUTNB

Topic: Divide and Conquer

83

Strassen's matrix multiplication:

- Discover a way to compute the C_{ij} 's using 7 multiplications and 18 additions or subtractions

$$P = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$Q = (A_{21} + A_{22})B_{11}$$

$$R = A_{11}(B_{12} - B_{22})$$

$$S = A_{22}(B_{21} - B_{11})$$

$$T = (A_{11} + A_{12})B_{22}$$

$$U = (A_{21} - A_{11})(B_{11} + B_{12})$$

$$V = (A_{12} - A_{22})(B_{21} + B_{22})$$

$$C_{11} = P + S - T + V$$

$$C_{12} = R + T$$

$$C_{21} = Q + S$$

$$C_{22} = P + R - Q + U$$

CIS KMUTNB

Topic: Divide and Conquer

84

•Algorithm :

procedure Strassen (n, A, B, C) // n is size, A, B the input
matrices, C output matrix

begin

if $n = 2$,

$$\left\{ \begin{array}{l} C_{11} = a_{11} \cdot b_{11} + a_{12} \cdot b_{21}; \\ C_{12} = a_{11} \cdot b_{12} + a_{12} \cdot b_{22}; \\ C_{21} = a_{12} \cdot b_{11} + a_{22} \cdot b_{21}; \\ C_{22} = a_{12} \cdot b_{12} + a_{22} \cdot b_{22}; \end{array} \right.$$

else

(cont.)

else

Partition A into 4 submatrices: $A_{11}, A_{12}, A_{21}, A_{22}$;
Partition B into 4 submatrices: $B_{11}, B_{12}, B_{21}, B_{22}$;
call Strassen ($\frac{n}{2}, A_{11} + A_{22}, B_{11} + B_{22}, P$) ;

call Strassen ($\frac{n}{2}, A_{21} + A_{22}, B_{11}, Q$);

call Strassen ($\frac{n}{2}, A_{11}, B_{12} - B_{22}, R$);

call Strassen ($\frac{n}{2}, A_{22}, B_{21} - B_{11}, S$);

call Strassen ($\frac{n}{2}, A_{11} + A_{12}, B_{22}, T$);

```

    call Strassen ( $\frac{n}{2}, A_{21} - A_{11}, B_{11} + B_{12}, U$ );
    call Strassen ( $\frac{n}{2}, A_{21} - A_{11}, B_{11} + B_{12}, U$ );
     $C_{11} = P + S - T + V$ ;
     $C_{12} = R + T$ ;
     $C_{21} = Q + S$ ;
     $C_{22} = P + R - Q + U$ ;
end;

```

Analysis:

$$T(n) = \begin{cases} 7 \cdot T(\frac{n}{2}) + an^2 & n > 2 \\ b & n \leq 2 \end{cases}$$

$$\begin{aligned}
 T(n) &= 7 \cdot T(\frac{n}{2}) + an^2 \\
 &= 7^2 \cdot T(\frac{n}{2^2}) + (\frac{7}{4})an^2 + an^2 \\
 &= 7^3 \cdot T(\frac{n}{2^3}) + (\frac{7}{4})^2 \cdot an^2 + (\frac{7}{4}) \cdot an^2 + an^2 \\
 &\quad \dots
 \end{aligned}$$

Assume $n = 2^k$ for some integer k

$$\begin{aligned}
 &= 7^{k-1} \cdot T\left(\frac{n}{2^{k-1}}\right) + an^2 \cdot \left[\left(\frac{7}{4}\right)^{k-2} + \dots + 1 \right] \\
 &= 7^{k-1} \cdot b + an^2 \left[\frac{\left(\frac{7}{4}\right)^{k-1} - 1}{\frac{7}{4} - 1} \right] \\
 &\leq b \cdot 7^k + c \cdot n^2 \cdot \left(\frac{7}{4}\right)^k \\
 &= b \cdot 7^{Lgn} + cn^2 \cdot \left(\frac{7}{4}\right)^{Lgn} = b \cdot 7^{Lgn} + cn^2 (n)^{Lg \frac{7}{4}} \\
 &= b \cdot n^{Lg 7} + cn^{Lg 7} = (b+c) \cdot n^{Lg 7} \\
 &= O(n^{Lg 7}) = O(n^{2.81})
 \end{aligned}$$

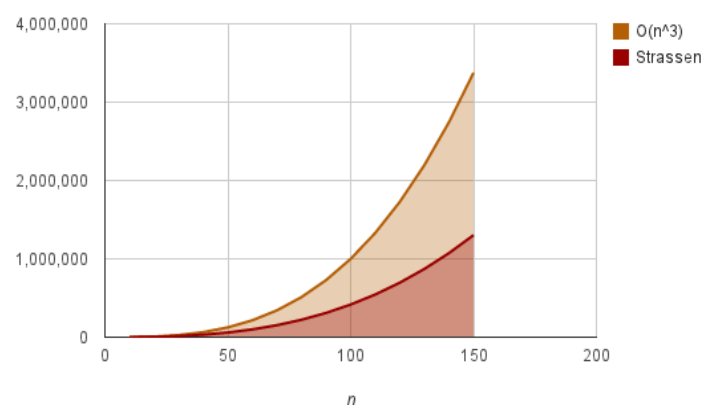
CIS KMUTNB

Topic: Divide and Conquer

89

Strassen's Matrix Multiplication

เปรียบเทียบจำนวนครั้งในการคูณเมื่อ n เพิ่มขึ้น



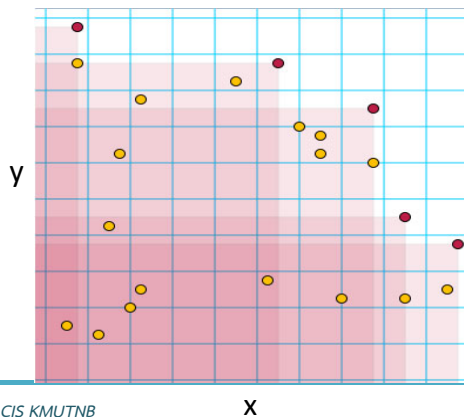
CIS KMUTNB

90

Maxima set problem

ในระนาบ X,Y กำหนดให้มี n จุด จุด p คือ **maximal** หากไม่มีจุด q ใดๆ ที่สามารถครอบคลุม (dominated) p ได้

หมายเหตุ p จะไม่ถูกครอบคลุมโดย q หาก $(p.x > q.x)$ และ $(p.y > q.y)$



ดังนั้นปัญหานี้ก็คือการออกแบบอัลกอริทึมที่มีประสิทธิภาพในการค้นหาเซตของ maximum points

CIS KMUTNB

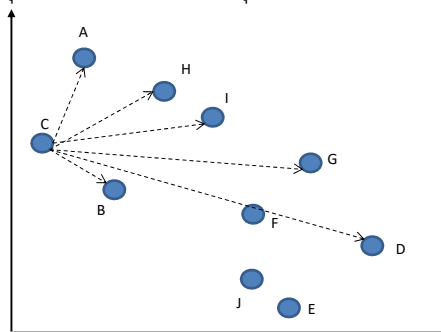
X

91

Maxima set problem with Brute force

หลักการ brute force ก็คือไล่เช็คทีละจุด โดยหากพบว่ามียังน้อย 1 จุดที่ถูกครอบคลุมโดยจุดใดๆ ก็จะถือว่าจุดดังกล่าวไม่ได้อยู่ในเซตคำตอบ

ดังนั้นหากมีทั้งหมด n จุด คาดหวังได้ว่าจำนวนจุดที่ต้องเปรียบเทียบทั้งหมดคือ n^2

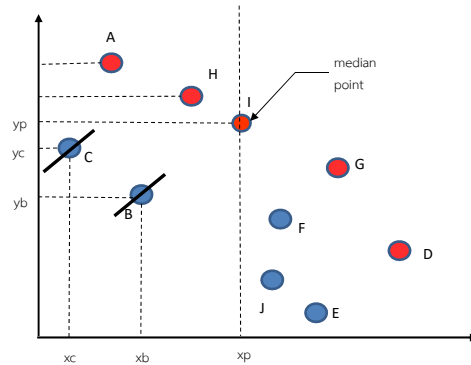


CIS KMUTNB

92

Divide & Conquer Solution

1. เรียงลำดับจุดทั้งหมดสอดคล้องสัมพันธ์กับค่า x (lexicographic order)
2. ค้นหาจุดกลาง (median point) เป็นจุดอ้างอิงและเป็นตัวแบ่งกลุ่มออกเป็น 2 กลุ่ม
3. ทุกๆ จุดด้านซ้ายที่มีค่า y มากกว่า y_p คือ maxima point
4. ทุกๆ จุดด้านขวาเป็น maxima point ยกเว้นแต่จะมีบางจุดด้านขวาที่มีค่า y มากกว่า



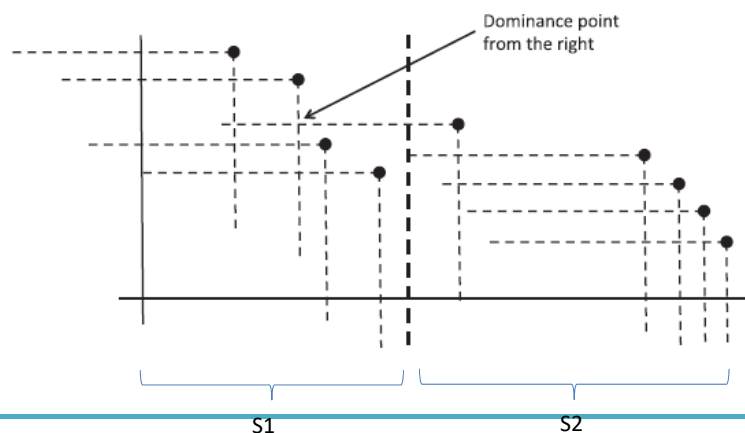
CIS KMUTNB

93

Divide & Conquer Solution

อย่างไรก็ตามอาจมีบางจุดที่เป็น maxima ด้านซ้าย S1 ถูกครอบคลุมโดยบางจุดที่เป็น maxima ด้านขวา S2

วิธีการแก้ปัญหาก็คือทุกครั้งที่มีการ add จุดเข้าไปที่ S2 จะต้องนำค่า y ของจุดนั้นไปเช็คกับทุกๆ จุด maxima ใน S1 และกำจัดทิ้งหากพบว่าค่า y ของจุดใน S1 มีค่าน้อยกว่า



CIS KMUTNB

94

Divide & Conquer Solution

ประมวลผลเบื้องต้น เรียงลำดับจุดทั้งหมดสอดคล้องสัมพันธ์กับค่า x (lexicographic order)

Base case: ถ้า $n = 1$ เพิ่มจุดเข้าไปในเซตคำตอบ S

Divide: ให้ p เป็นจุดกลาง (median) ดังนั้นแบ่งจุดที่เหลือออกเป็น 2 กลุ่มคือ S_1 และ S_2
โดยทุกๆ จุดใน S_1 จะมีค่า x น้อยกว่า x_p มิฉะนั้นจะถูกเก็บไว้ใน S_2

Combine:

** ให้ q เป็นจุดที่มีค่า x ต่ำสุดใน S_2 จากนั้นกำจัดทุกๆ จุด ใน S_1 ที่มีค่า y น้อยกว่า y_q
เซตคำตอบคือ $S = S_1 \cup S_2$

CIS KMUTNB

95

Solution

Algorithm MaximaSet(S):

Input: A set, S , of n points in the plane

Output: The set, M , of maxima points in S

if $n \leq 1$ **then**

return S

Let p be the median point in S , by lexicographic (x, y) -coordinates

Let L be the set of points lexicographically less than p in S

Let G be the set of points lexicographically greater than or equal to p in S

$M_1 \leftarrow \text{MaximaSet}(L)$

$M_2 \leftarrow \text{MaximaSet}(G)$

Let q be the lexicographically smallest point in M_2

for each point, r , in M_1 **do**

if ~~$x(r) \leq x(q)$~~ **and** $y(r) \leq y(q)$ **then**

 Remove r from M_1

return $M_1 \cup M_2$

CIS KMUTNB

96

Counting Inversion

- Given $A[n] = \{a_1, a_2, \dots, a_n\}$
 - Inversions i and j
 - if $i < j$ but $a_i > a_j$
- Example
 - $A = \{1, 3, 4, 2, 5\}$
 - There are 2 inversions = 3-2, 4-2
- Brute force: check all $O(n^2)$ pairs i and j

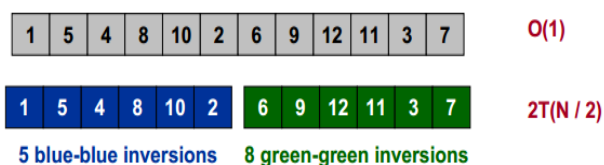
CIS KMUTNB

Divide and Conquer

97

Counting Inversion: Divide and Conquer algorithm

1. Divide: separate list into two pieces.
2. Conquer: recursively count inversions in each half.
3. **Combine: count inversions where a_i and a_j in different halves**



How to Combine?

CIS KMUTNB

Divide and Conquer

98